

Artificial Intelligence & Machine Learning Course

Generative AI

DR. JOYDEEP CHANDRA
ASSOCIATE PROFESSOR
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY PATNA, INDIA

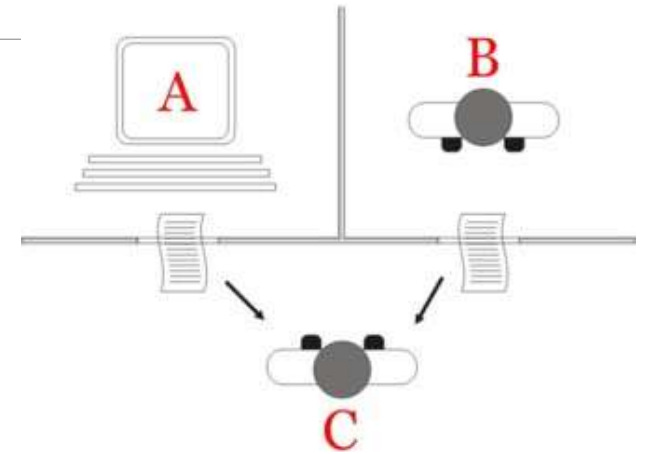


What is AI?

- "... the science and engineering of making intelligent machines" (John McCarthy)
- AI is intelligence demonstrated by machines, in contrast to the natural intelligence displayed by humans. (Wikipedia)
- AI attempts not just to understand but also to build intelligent entities. (Russell & Norvig)
- Artificial intelligence is that activity devoted to making machines intelligent, and intelligence is that quality that enables an entity to function appropriately and with foresight in its environment. (Nilsson)

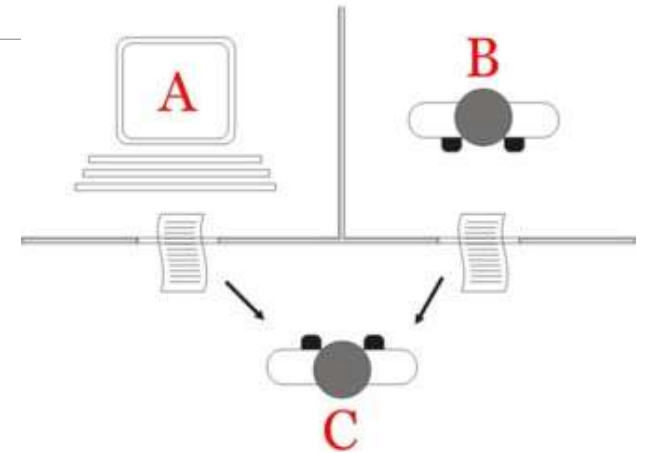
What is AI?

- Act like human – The 'Turing test'
 - A human interrogator fails to distinguish responses generated from a computer or a human



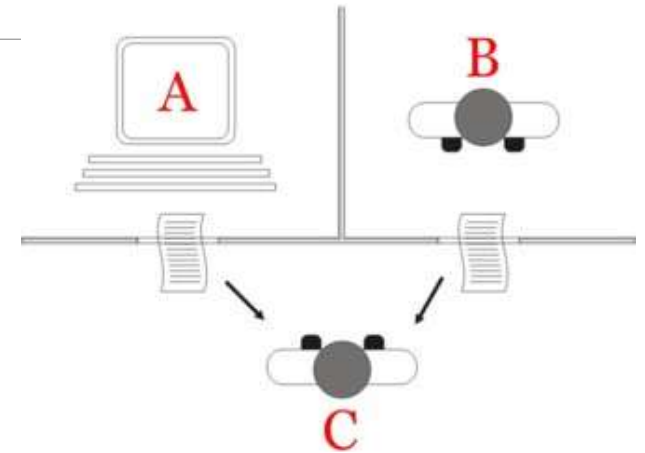
What is AI?

- **Act like human** – The 'Turing test'
 - A human interrogator fails to distinguish responses generated from a computer or a human
- **Think like human** – Cognitive modeling
 - Need to know how human brain functions - through introspection, psychological experiments, brain imaging, etc.



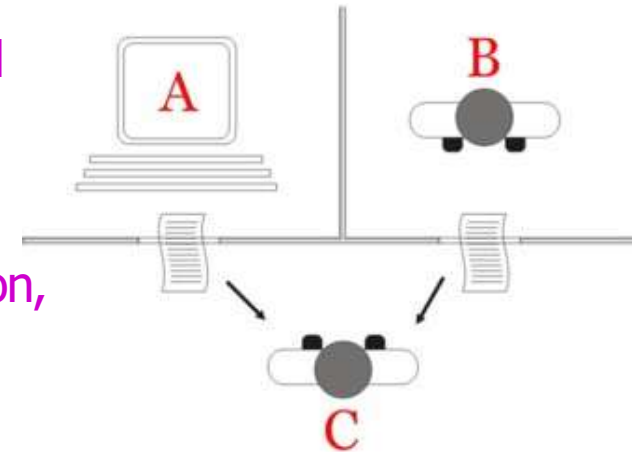
What is AI?

- **Act like human** – The 'Turing test'
 - A human interrogator fails to distinguish responses generated from a computer or a human
- **Think like human** – Cognitive modeling
 - Need to know how human brain functions - through introspection, psychological experiments, brain imaging, etc.
- **Think rationally** – Laws of thought
 - Right thinking, irrefutable reasoning processes

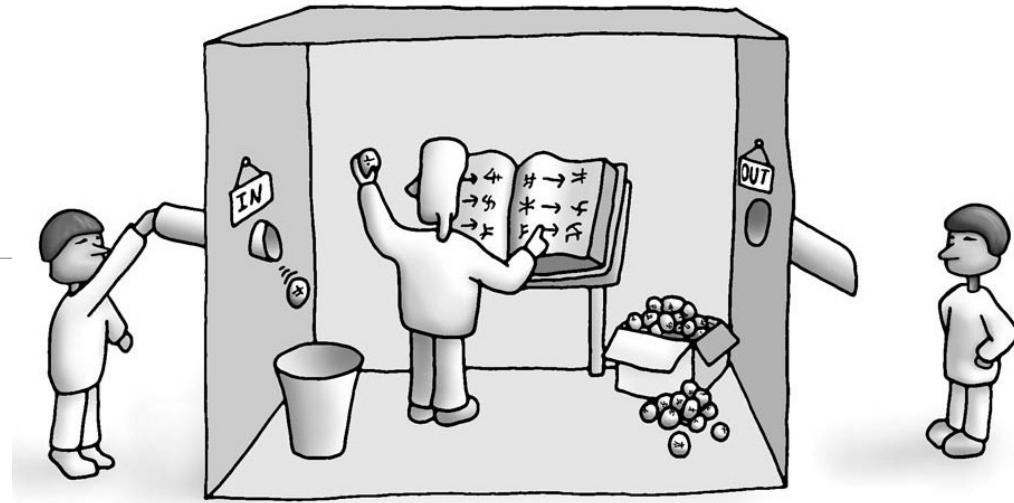


What is AI?

- **Act like human** – The 'Turing test'
 - A human interrogator fails to distinguish responses generated from a computer or a human
- **Think like human** – Cognitive modeling
 - Need to know how human brain functions - through introspection, psychological experiments, brain imaging, etc.
- **Think rationally** – Laws of thought
 - Right thinking, irrefutable reasoning processes
- **Act rationally** – Rational agent
 - Acts to achieve the best outcome in presence of uncertainty, the best expected outcome

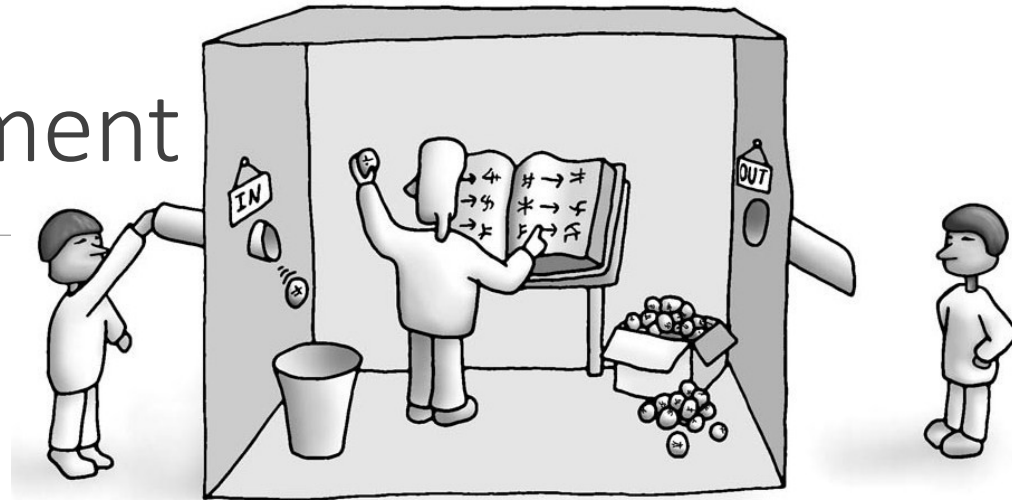


Chinese room argument



- Suppose, artificial intelligence has succeeded in constructing a computer that behaves as if it understands Chinese. It takes Chinese characters as input and, by following the instructions of a computer program, produces other Chinese characters, which it presents as output.
 - Suppose, computer performs its task and comfortably passes the Turing test

Chinese room argument



- Suppose, artificial intelligence has succeeded in constructing a computer that behaves as if it understands Chinese. It takes Chinese characters as input and, by following the instructions of a computer program, produces other Chinese characters, which it presents as output.
 - Suppose, computer performs its task and comfortably passes the Turing test
 - Does the machine literally “understand” Chinese? – Strong AI
 - Is it merely simulating the ability to understand Chinese? – Weak AI

General goals AI for intelligent agent

- Perception - view about the environment
- Reasoning - logical analysis
- Control / Motion / Manipulation - maneuvering
- Planning - sequence of steps
- Communication - interaction with others through language or other mediums
- Creativity - synthesizing new things
- Learning - understanding

General goals GenAI

- Communication - comprehension and generation of human language, LLM works well
- Perception - visual question answering, multimodal foundational models
- Reasoning - finding a proof, chain of thought
- Control / Motion / Manipulation - fine grain control for complex environment, quadruped robot
- Planning - LLMs are good at planning
- Creativity - text-to-image, text-to-music
- Learning - in context learning (provide examples as context)

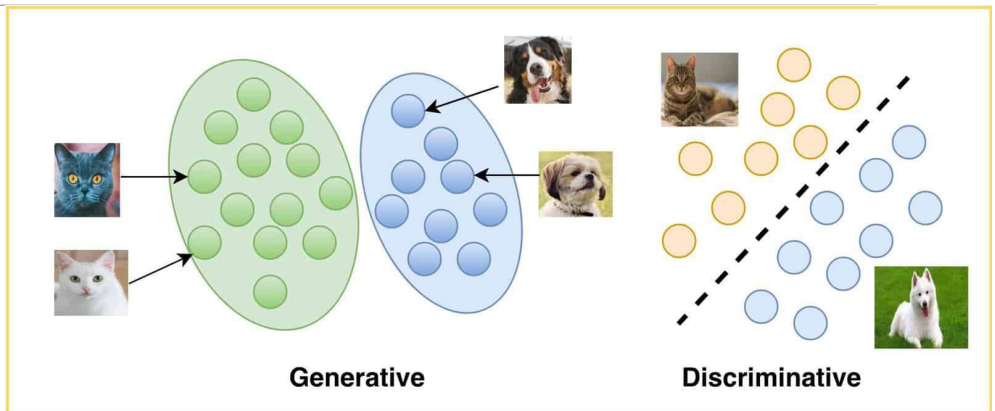
Overview

- Topic: Introduction to Generative Models
- Goal: Understand the basics, types, and applications
- Tools: TensorFlow

Part 1: Generative model concepts

What is a Generative Model

- A model that learns the underlying distribution of data
 - They learn the **joint probability** of input and label $P(x,y)$ or just $P(x)$
 - Purpose is to **Generate** new data similar to training data; can also classify
 - Models how the data is generated
 - Can generate new data samples similar to the training data
- Limitations
 - Often complex and more difficult to train



Suppose you have a dataset that has one feature

- The skin color of a person (X)

Each instance label is the continent which the person belongs (Y)

$P(Y|X)$: Gives the

- Prob that a given person with color X belongs to continent Y
- Can be used to classify the continent for a given skin color

$P(X,Y) = P(X|Y)P(Y)$: Gives the

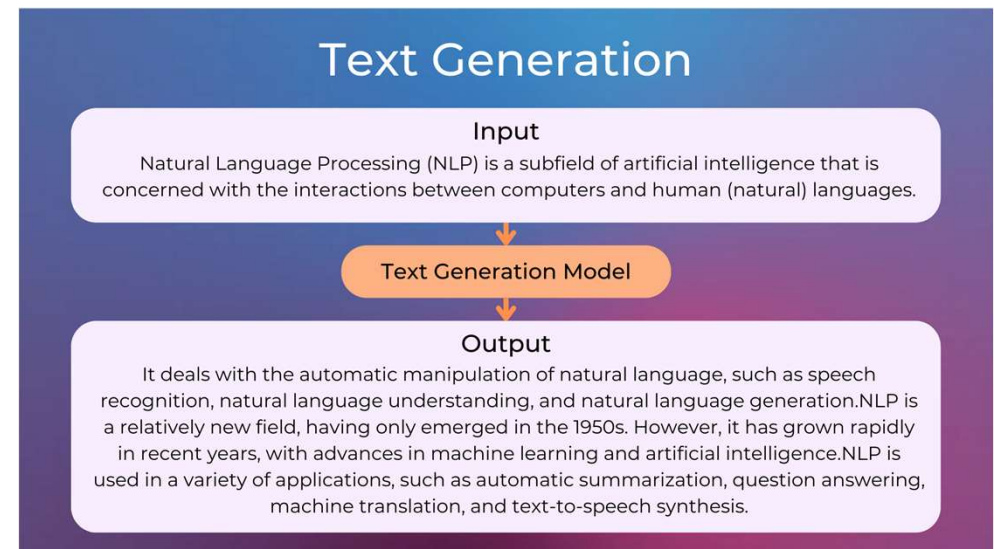
- Prob of a random person belonging to continent and
- Given a person from continent Y , the prob of his skin color
- Can be used to generate the whole population

Discriminative vs. Generative Model

Generative Models	Discriminative Models
Model the joint distribution $P(x,y)$	Model the conditional distribution ($P(y x)$)
Learn how data is generated	Learn how to separate classes
Can generate new data	Cannot generate data
Models the full data distribution	Models only the decision boundary
Training complexity may be high	Lower training complexity
Better handles missing inputs	Less robust to missing data
Naive Bayes, Gaussian Mixture Models, GANs, VAEs	Logistic Regression, SVMs, Decision Trees, BERT
Works well with small data, but with prior knowledge	May work well on small data depending upon feature quality
Mainly used for Data generation, imputation, anomaly detection	Can be used for several downstream tasks like Spam detection, sentiment analysis, image classification

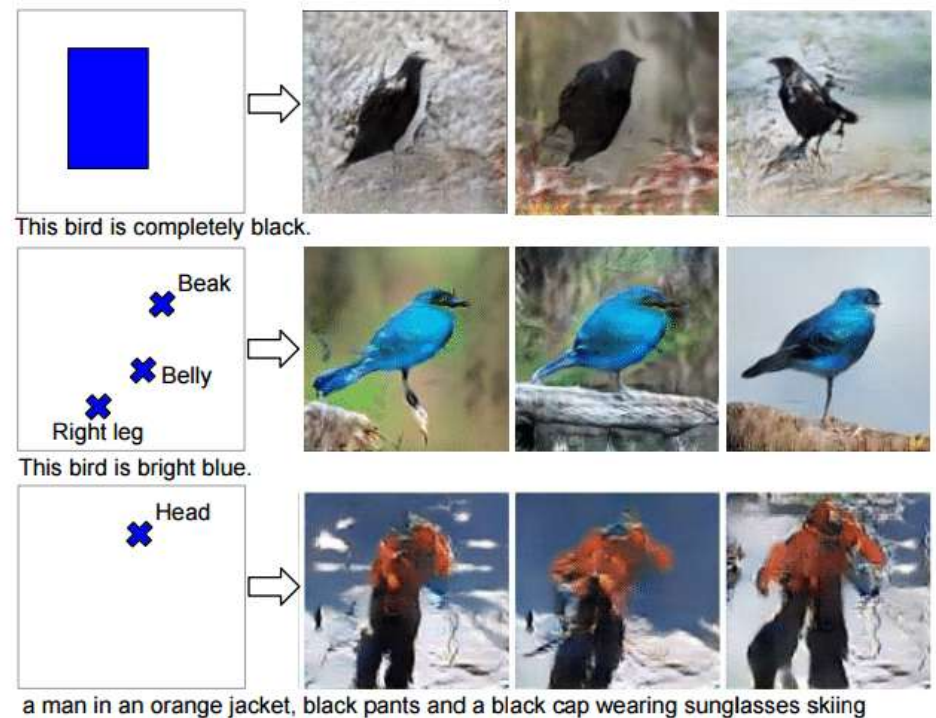
Applications of Generative models

- **Text generation (e.g., GPT)**
- Image synthesis (e.g., GANs)
- Music and audio generation
- Data augmentation and imputation



Applications of Generative models

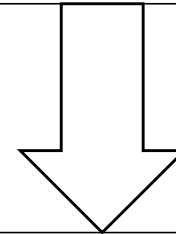
- Text generation (e.g., GPT)
- **Image synthesis (e.g., GANs)**
- Music and audio generation
- Data augmentation and imputation



Applications of Generative models

- Text generation (e.g., GPT)
- Image synthesis (e.g., GANs)
- **Music and audio generation**
- Data augmentation and imputation

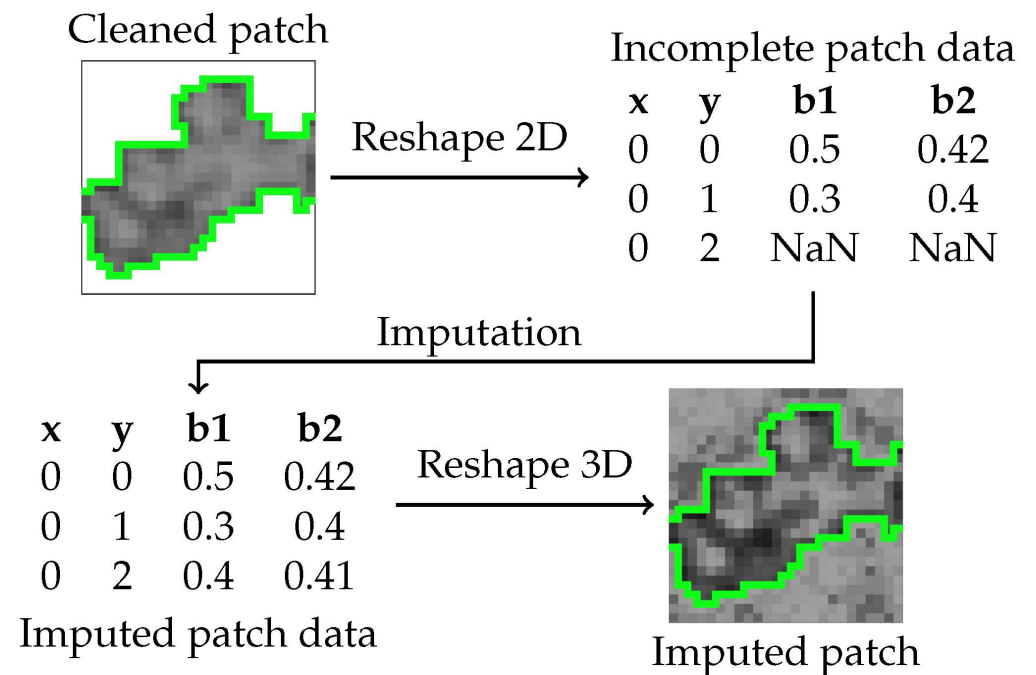
The songs I sing, the future I dream,
what would it take to get a world,
where we love each other to the brim.



The Song

Applications of Generative models

- Text generation (e.g., GPT)
- Image synthesis (e.g., GANs)
- Music and audio generation
- **Data augmentation and imputation**



Categories of Generative Models

- Explicit density models
- Implicit models
- Diffusion based models
- Autoregressive models

Explicit Density Models

Explicit density models are generative models that

- attempt to learn and represent the probability distribution $P(x)$ (or $P(x|y)$, etc.) directly and explicitly.

Example models: Naïve Bayes, Gaussian Mixture Models

Bayes Classifier

A probabilistic framework for solving classification problems

Conditional Probability: $P(Y | X) = \frac{P(X, Y)}{P(X)}$

$$P(X | Y) = \frac{P(X, Y)}{P(Y)}$$

Bayes theorem: $P(Y | X) = \frac{P(X | Y)P(Y)}{P(X)}$

Using Bayes Theorem for Classification

Consider each attribute and class label as random variables

Given a record with attributes $(X_1, X_2, \dots, X_d)$, the goal is to predict class Y

- Specifically, we want to find the value of Y that maximizes $P(Y | X_1, X_2, \dots, X_d)$

Can we estimate $P(Y | X_1, X_2, \dots, X_d)$ directly from data?

<i>Tid</i>	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Using Bayes Theorem for Classification

Approach:

- compute posterior probability $P(Y \mid X_1, X_2, \dots, X_d)$ using the Bayes theorem

$$P(Y \mid X_1 X_2 \dots X_n) = \frac{P(X_1 X_2 \dots X_d \mid Y) P(Y)}{P(X_1 X_2 \dots X_d)}$$

- *Maximum a-posteriori*: Choose Y that maximizes $P(Y \mid X_1, X_2, \dots, X_d)$
- Equivalent to choosing value of Y that maximizes $P(X_1, X_2, \dots, X_d \mid Y) P(Y) = P(X, Y) \rightarrow$ That's why Naïve Bayes is generative

How to estimate $P(X_1, X_2, \dots, X_d \mid Y)$?

Example Data

Given a Test Record:

<i>Tid</i>	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

$X = (\text{Refund} = \text{No}, \text{Divorced}, \text{Income} = 120\text{K})$

- We need to estimate
 $P(\text{Evade} = \text{Yes} \mid X)$ and $P(\text{Evade} = \text{No} \mid X)$

In the following we will replace

Evade = Yes by Yes, and

Evade = No by No

Example Data

Given a Test Record:

Tid	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

$X = (\text{Refund} = \text{No}, \text{Divorced}, \text{Income} = 120\text{K})$

Using Bayes Theorem:

$$\square P(\text{Yes} | X) = \frac{P(X | \text{Yes})P(\text{Yes})}{P(X)}$$

$$\square P(\text{No} | X) = \frac{P(X | \text{No})P(\text{No})}{P(X)}$$

$\square$ How to estimate $P(X | \text{Yes})$ and $P(X | \text{No})$?

Conditional Independence

X and Y are conditionally independent given Z if $P(X|YZ) = P(X|Z)$

Example: Arm length and reading skills

- Young child has shorter arm length and limited reading skills, compared to adults
- If age is fixed, no apparent relationship between arm length and reading skills
- Arm length and reading skills are conditionally independent given age

Naïve Bayes Classifier

Assume independence among attributes X_i when class is given:

- $P(X_1, X_2, \dots, X_d | Y_j) = P(X_1 | Y_j) P(X_2 | Y_j) \dots P(X_d | Y_j)$
- Now we can estimate $P(X_i | Y_j)$ for all X_i and Y_j combinations from the training data
- New point is classified to Y_j if $P(Y_j) \prod P(X_i | Y_j)$ is maximal.

Naïve Bayes on Example Data

Given a Test Record:

<i>Tid</i>	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

$X = (\text{Refund} = \text{No}, \text{Divorced}, \text{Income} = 120\text{K})$

$P(X | \text{Yes}) =$

$P(\text{Refund} = \text{No} | \text{Yes}) \times$

$P(\text{Divorced} | \text{Yes}) \times$

$P(\text{Income} = 120\text{K} | \text{Yes})$

$P(X | \text{No}) =$

$P(\text{Refund} = \text{No} | \text{No}) \times$

$P(\text{Divorced} | \text{No}) \times$

$P(\text{Income} = 120\text{K} | \text{No})$

Estimate Probabilities from Data

Tid	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

$P(y)$ = fraction of instances of class y

— e.g., $P(\text{No}) = 7/10$,
 $P(\text{Yes}) = 3/10$

For categorical attributes:

$$P(X_i = c \mid y) = n_c / n$$

— where $|X_i = c|$ is number of instances having attribute value $X_i = c$ and belonging to class y

— Examples:

$$P(\text{Status} = \text{Married} \mid \text{No}) = 4/7$$

$$P(\text{Refund} = \text{Yes} \mid \text{Yes}) = 0$$

Estimate Probabilities from Data

For continuous attributes:

- **Discretization:** Partition the range into bins:
 - ◆ Replace continuous value with bin value
 - Attribute changed from continuous to ordinal
- **Probability density estimation:**
 - ◆ Assume attribute follows a normal distribution
 - ◆ Use data to estimate parameters of distribution (e.g., mean and standard deviation)
 - ◆ Once probability distribution is known, use it to estimate the conditional probability $P(X_i|Y)$

Estimate Probabilities from Data

Tid	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Normal distribution:

$$P(X_i | Y_j) = \frac{1}{\sqrt{2\pi\sigma_{ij}^2}} e^{-\frac{(X_i - \mu_{ij})^2}{2\sigma_{ij}^2}}$$

— One for each (X_i, Y_i) pair

For (Income, Class=No):

— If Class=No

◆ sample mean = 110

◆ sample variance = 2975

$$P(\text{Income} = 120 | \text{No}) = \frac{1}{\sqrt{2\pi(54.54)}} e^{-\frac{(120-110)^2}{2(2975)}} = 0.0072$$

Example of Naïve Bayes Classifier

Given a Test Record: $X = (\text{Refund} = \text{No}, \text{Divorced}, \text{Income} = 120\text{K})$

Naïve Bayes Classifier:

$$P(\text{Refund} = \text{Yes} \mid \text{No}) = 3/7$$

$$P(\text{Refund} = \text{No} \mid \text{No}) = 4/7$$

$$P(\text{Refund} = \text{Yes} \mid \text{Yes}) = 0$$

$$P(\text{Refund} = \text{No} \mid \text{Yes}) = 1$$

$$P(\text{Marital Status} = \text{Single} \mid \text{No}) = 2/7$$

$$P(\text{Marital Status} = \text{Divorced} \mid \text{No}) = 1/7$$

$$P(\text{Marital Status} = \text{Married} \mid \text{No}) = 4/7$$

$$P(\text{Marital Status} = \text{Single} \mid \text{Yes}) = 2/3$$

$$P(\text{Marital Status} = \text{Divorced} \mid \text{Yes}) = 1/3$$

$$P(\text{Marital Status} = \text{Married} \mid \text{Yes}) = 0$$

For Taxable Income:

If class = No: sample mean = 110

sample variance = 2975

If class = Yes: sample mean = 90

sample variance = 25

- $$\begin{aligned} P(X \mid \text{No}) &= P(\text{Refund}=\text{No} \mid \text{No}) \\ &\times P(\text{Divorced} \mid \text{No}) \\ &\times P(\text{Income}=120\text{K} \mid \text{No}) \\ &= 4/7 \times 1/7 \times 0.0072 = 0.0006 \end{aligned}$$
- $$\begin{aligned} P(X \mid \text{Yes}) &= P(\text{Refund}=\text{No} \mid \text{Yes}) \\ &\times P(\text{Divorced} \mid \text{Yes}) \\ &\times P(\text{Income}=120\text{K} \mid \text{Yes}) \\ &= 1 \times 1/3 \times 1.2 \times 10^{-9} = 4 \times 10^{-10} \end{aligned}$$

Since $P(X \mid \text{No})P(\text{No}) > P(X \mid \text{Yes})P(\text{Yes})$

Therefore $P(\text{No} \mid X) > P(\text{Yes} \mid X)$

$\Rightarrow$ Class = No

Naïve Bayes Classifier can make decisions with partial information about attributes in the test record

Even in absence of information about any attributes, we can use Apriori Probabilities of Class Variable:

$$P(\text{Yes}) = 3/10$$

$$P(\text{No}) = 7/10$$

Naïve Bayes Classifier:

$$P(\text{Refund} = \text{Yes} \mid \text{No}) = 3/7$$

$$P(\text{Refund} = \text{No} \mid \text{No}) = 4/7$$

$$P(\text{Refund} = \text{Yes} \mid \text{Yes}) = 0$$

$$P(\text{Refund} = \text{No} \mid \text{Yes}) = 1$$

$$P(\text{Marital Status} = \text{Single} \mid \text{No}) = 2/7$$

$$P(\text{Marital Status} = \text{Divorced} \mid \text{No}) = 1/7$$

$$P(\text{Marital Status} = \text{Married} \mid \text{No}) = 4/7$$

$$P(\text{Marital Status} = \text{Single} \mid \text{Yes}) = 2/3$$

$$P(\text{Marital Status} = \text{Divorced} \mid \text{Yes}) = 1/3$$

$$P(\text{Marital Status} = \text{Married} \mid \text{Yes}) = 0$$

For Taxable Income:

If class = No: sample mean = 110

sample variance = 2975

If class = Yes: sample mean = 90

sample variance = 25

If we only know that marital status is Divorced, then:

$$P(\text{Yes} \mid \text{Divorced}) = 1/3 \times 3/10 / P(\text{Divorced})$$

$$P(\text{No} \mid \text{Divorced}) = 1/7 \times 7/10 / P(\text{Divorced})$$

If we also know that Refund = No, then

$$P(\text{Yes} \mid \text{Refund} = \text{No}, \text{Divorced}) = 1 \times 1/3 \times 3/10 / P(\text{Divorced}, \text{Refund} = \text{No})$$

$$P(\text{No} \mid \text{Refund} = \text{No}, \text{Divorced}) = 4/7 \times 1/7 \times 7/10 / P(\text{Divorced}, \text{Refund} = \text{No})$$

If we also know that Taxable Income = 120, then

$$P(\text{Yes} \mid \text{Refund} = \text{No}, \text{Divorced}, \text{Income} = 120) = 1.2 \times 10^{-9} \times 1 \times 1/3 \times 3/10 / P(\text{Divorced}, \text{Refund} = \text{No}, \text{Income} = 120)$$

$$P(\text{No} \mid \text{Refund} = \text{No}, \text{Divorced}, \text{Income} = 120) = 0.0072 \times 4/7 \times 1/7 \times 7/10 / P(\text{Divorced}, \text{Refund} = \text{No}, \text{Income} = 120)$$

Issues with Naïve Bayes Classifier

Given a Test Record: $X = (\text{Married})$

Naïve Bayes Classifier:

$$P(\text{Yes}) = 3/10$$

$$P(\text{No}) = 7/10$$

$$P(\text{Refund} = \text{Yes} \mid \text{No}) = 3/7$$

$$P(\text{Refund} = \text{No} \mid \text{No}) = 4/7$$

$$P(\text{Refund} = \text{Yes} \mid \text{Yes}) = 0$$

$$P(\text{Refund} = \text{No} \mid \text{Yes}) = 1$$

$$P(\text{Yes} \mid \text{Married}) = 0 \times 3/10 / P(\text{Married})$$

$$P(\text{Marital Status} = \text{Single} \mid \text{No}) = 2/7$$

$$P(\text{Marital Status} = \text{Divorced} \mid \text{No}) = 1/7$$

$$P(\text{Marital Status} = \text{Married} \mid \text{No}) = 4/7$$

$$P(\text{Marital Status} = \text{Single} \mid \text{Yes}) = 2/3$$

$$P(\text{Marital Status} = \text{Divorced} \mid \text{Yes}) = 1/3$$

$$P(\text{Marital Status} = \text{Married} \mid \text{Yes}) = 0$$

$$P(\text{No} \mid \text{Married}) = 4/7 \times 7/10 / P(\text{Married})$$

For Taxable Income:

If class = No: sample mean = 110

sample variance = 2975

If class = Yes: sample mean = 90

sample variance = 25

Issues with Naïve Bayes Classifier

Consider the table with Tid = 7 deleted

Tid	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Given X = (Refund = Yes, Divorced, 120K)

$$P(X \mid \text{No}) = 2/6 \times 0 \times 0.0083 = 0$$

$$P(X \mid \text{Yes}) = 0 \times 1/3 \times 1.2 \times 10^{-9} = 0$$

Naïve Bayes Classifier:

$$P(\text{Refund} = \text{Yes} \mid \text{No}) = 2/6$$

$$P(\text{Refund} = \text{No} \mid \text{No}) = 4/6$$

$$P(\text{Refund} = \text{Yes} \mid \text{Yes}) = 0$$

$$P(\text{Refund} = \text{No} \mid \text{Yes}) = 1$$

$$P(\text{Marital Status} = \text{Single} \mid \text{No}) = 2/6$$

$$P(\text{Marital Status} = \text{Divorced} \mid \text{No}) = 0$$

$$P(\text{Marital Status} = \text{Married} \mid \text{No}) = 4/6$$

$$P(\text{Marital Status} = \text{Single} \mid \text{Yes}) = 2/3$$

$$P(\text{Marital Status} = \text{Divorced} \mid \text{Yes}) = 1/3$$

$$P(\text{Marital Status} = \text{Married} \mid \text{Yes}) = 0/3$$

For Taxable Income:

If class = No: sample mean = 91

sample variance = 685

If class = No: sample mean = 90

sample variance = 25

Naïve Bayes will not be able to
classify X as Yes or No!

Issues with Naïve Bayes Classifier

If one of the conditional probabilities is zero, then the entire expression becomes zero

Need to use other estimates of conditional probabilities than simple fractions

Probability estimation:

$$\text{original: } P(X_i = c|y) = \frac{n_c}{n}$$

$$\text{Laplace Estimate: } P(X_i = c|y) = \frac{n_c + 1}{n + v}$$

$$\text{m - estimate: } P(X_i = c|y) = \frac{n_c + mp}{n + m}$$

n : number of training
instances belonging to class y

n_c : number of instances with
 $X_i = c$ and $Y = y$

v : total number of attribute
values that X_i can take

p : initial estimate of
($P(X_i = c|y)$ known apriori

m : hyper-parameter for our
confidence in p

Implementing Naïve Bayes

Loading Libraries

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report
```

Load the dataset

```
df = pd.read_csv("Cards.csv")
```

Separate features and target

```
X = df.drop("Defaultee", axis=1)
```

```
y = df["Defaultee"]
```


Distinguish data types for different processing

```
numeric_features = [  
    "Income", "Limit", "Rating", "Cards",  
    "Age", "Education", "Balance"  
]  
categorical_features = [  
    "Gender", "Student", "Married", "Ethnicity"  
]
```

If you want to distinguish data types automatically then do the following:

```
import numpy as np  
numeric_features = X.select_dtypes(include=[np.number]).columns  
categorical_features = X.select_dtypes(exclude=[np.number]).columns
```

Transformation based on data type

```
numeric_transformer = StandardScaler()
```

```
categorical_transformer = OneHotEncoder(drop="first")
```

Combine preprocessing using column transformer

```
preprocessor = ColumnTransformer(  
    transformers=[  
        ("num", numeric_transformer, numeric_features),  
        ("cat", categorical_transformer, categorical_features)  
    ]  
)
```

Build the Naïve Bayes pipeline

```
model = Pipeline(steps=[  
    ("preprocessing", preprocessor),  
    ("classifier", GaussianNB())  
])
```

Split into train-test

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.25, random_state=42  
)
```

Train the model

```
model.fit(X_train, y_train)
```

Make predictions

```
y_pred = model.predict(X_test)
```

Evaluate model

```
print("Accuracy:", accuracy_score(y_test, y_pred))  
print(classification_report(y_test, y_pred))
```

Accuracy: 0.92

	precision	recall	f1-score	support
0	0.97	0.93	0.95	84
1	0.70	0.88	0.78	16
accuracy			0.92	100
macro avg	0.84	0.90	0.86	100
weighted avg	0.93	0.92	0.92	100

$$\text{Accuracy} = \frac{TP+T}{TP+FP+TN+FN}$$

$$\text{Precision for class 0 is } \Pr(0) = \frac{TP(0)}{TP(0)+FP(0)}$$

$$\text{Precision for class 1 is } \Pr(1) = \frac{TP(1)}{TP(1)+FP(1)}$$

$$\text{Macro average is } \frac{\Pr(0)+\Pr(1)}{2}$$

$$\text{Weighted average is } \Pr(0) \times \left(\frac{N(0)}{N}\right) + \Pr(1) \times \left(\frac{N(1)}{N}\right)$$

Inferencing on unseen data and printing results

Assume new data is in a file “Unseen_Data.csv”, with no Defaultee column

```
new_customers = pd.read_csv("Unseen_Data.csv")
predictions = model.predict(new_customers) # Make predictions, ND=0, D=1
probabilities = model.predict_proba(new_customers)
```

```
for i, pred in enumerate(predictions):
    if pred==1:
        status="Defaulter"
    else:
        status="Non-Defaulter"
    print(f"Customer {i+1}: {status}, Probability of default = {probabilities[i][1]:.2f}")
```

Naïve Bayes (Summary)

Robust to isolated noise points

Handle missing values by ignoring the instance during probability estimate calculations

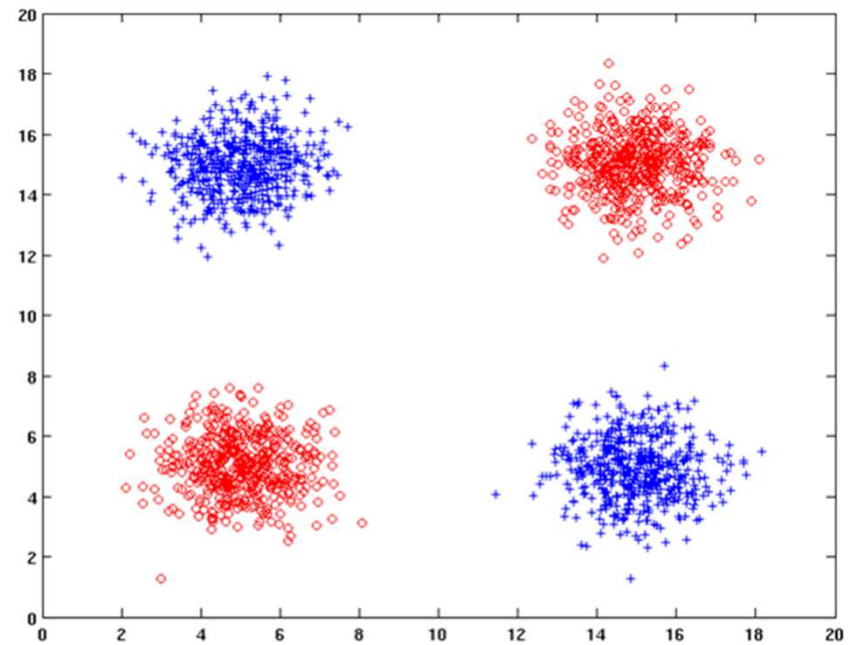
Robust to irrelevant attributes

Redundant and correlated attributes will violate class conditional assumption

- Use other techniques such as Bayesian Belief Networks (BBN)

Naïve Bayes

How does Naïve Bayes perform on the following dataset?



Conditional independence of attributes is violated

Explicit Density Models

Explicit density models are generative models that

- attempt to learn and represent the probability distribution $P(x)$ (or $P(x|y)$, etc.) directly and explicitly.

These models:

- Define a likelihood function $P(x|\theta)$, where θ are the parameters of the model.

Use this likelihood to train the model via

- maximum likelihood estimation (MLE) or variational inference.

Using Maximum Likelihood in a General Setting

Understanding how $P(x|\theta)$ can be estimated from a given data using Maximum Likelihood

Suppose we have the data given as $x = \{2.1, 2.5, 1.9, 2.2, 2.4\}$

We assume that the data comes from a normal distribution given as

$$P(x|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

Here $\theta = (\mu, \sigma^2)$ are the parameters of the distribution that we would like to estimate from the data

Using maximum likelihood

For the given data $x = \{2.1, 2.5, 1.9, 2.2, 2.4\}$ we define the likelihood function as

$$\begin{aligned} L &= \prod_{i=1}^5 P(x_i | \mu, \sigma^2) \\ &= \prod_{i=1}^5 \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x_i - \mu)^2}{2\sigma^2}\right) \end{aligned}$$

Taking log on both sides we get

$$\log L = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

Find μ and σ^2 that maximizes this $\log L$ term

How to maximize $\log L$ for the given data

$$\log L = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \mu)^2$$

Maximizing the likelihood

To maximize the likelihood we derive $\frac{\partial \log L}{\partial \mu}$ and $\frac{\partial \log L}{\partial \sigma^2}$ and set them to zero

While find the derivative wrt to μ we consider all other variables as fixed

- Similarly deriving wrt σ^2 we consider all other variables to be fixed

On deriving we get

$$\begin{aligned} \frac{\partial \log L}{\partial \mu} &= -\frac{1}{2\sigma^2} 2 \sum_{i=1}^n (x_i - \mu) = 0 \\ \Rightarrow \hat{\mu} &= \frac{1}{n} \sum_{i=1}^n x_i \end{aligned}$$

Estimate $\hat{\mu}$ and $\hat{\sigma}^2$ given the data set
 $x = \{2.1, 2.5, 1.9, 2.2, 2.4\}$

$$\hat{\mu} = 2.22$$

$$\hat{\sigma}^2 \approx 0.0416$$

Similarly we get

$$\hat{\sigma}^2 = \frac{1}{n} \sum_i^n (x_i - \hat{\mu})^2$$

Now we can generate any x_i with probability given as $\mathcal{N}(x_i | \hat{\mu}, \hat{\sigma}^2)$

Implementing MLE based model for generating images



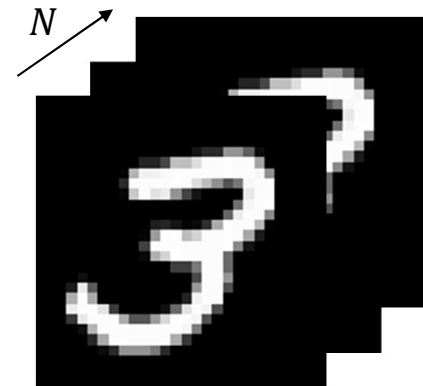
MNIST Dataset

Create stack of N , 2D images

Assume i^{th} pixel x_i is generated from a normal distribution $\mathcal{N}(\hat{\mu}_i, \hat{\sigma}_i^2)$

Estimate

$$\hat{\mu}_i = \frac{1}{N} \sum_{j=1}^N x_i^{(j)}$$
$$\hat{\sigma}_i^2 = \frac{1}{N} \sum_{j=1}^N (x_i^{(j)} - \hat{\mu}_i)^2$$



Generate a pixel $\hat{x}_i$ with probability $P(\hat{x}_i | \hat{\mu}_i, \hat{\sigma}_i^2)$

Try to run this and see the output

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist

(x_train, _), (_, _) = mnist.load_data()

x_train = x_train.astype(np.float32) / 255.0

x_train_flat = x_train.reshape(x_train.shape[0],
-1)

mu = np.mean(x_train_flat, axis=0)
sigma = np.std(x_train_flat, axis=0)

num_samples = 5
```

```
generated_samples = np.random.normal(
    loc=mu,
    scale=sigma,
    size=(num_samples, 784)
)
generated_samples = np.clip(generated_samples, 0.0,
1.0)
generated_images =
generated_samples.reshape(num_samples, 28, 28)

plt.figure(figsize=(10, 2))

for i in range(num_samples):
    plt.subplot(1, num_samples, i + 1)
    plt.imshow(generated_images[i], cmap="gray")
    plt.axis("off")
plt.show()
```

Limitations of the simple MLE model

Assumption of a model distribution

- MLE requires assumption of a known distribution (e.g., Gaussian).
- Real-world data is often multi-modal, non-Gaussian, or structured, so simple parametric forms may not fit well

Poor performance in higher dimension

- For high-dimensional data (e.g., images), computing and storing full covariance matrices becomes impractical.
- Pixel-wise independence assumptions (e.g., in naive Gaussian MLE for images) ignore spatial correlations and structure

Sensitivity to outliers

- MLE is highly sensitive to outliers because it tries to maximize the total likelihood, and outliers can dominate the log-likelihood function.

Limitations of the simple MLE model

Overfitting in the case of small data

- With few observations and many parameters, MLE tends to overfit—fitting noise instead of general trends.

Cannot capture complex dependencies

- MLE with simple models (e.g., Gaussian) cannot capture dependencies between features (like pixel relationships in images or syntax in language).

When is MLE based density estimation suitable

For simple, low-dimensional datasets.

When the data fits well with known distributions.

As a starting point for probabilistic modeling and understanding density estimation.

Implicit Generative Models

- No explicit density function
- Learn to generate samples from data distribution
- Example: Generative Adversarial Networks (GANs)